Сколько на самом деле стоят leveling и tiering: write, read и space amplification LSM-дерева против учебных оценок и роль бит на ключ в фильтре Блума

1. Вопрос

В LSM-дереве [1] записи копятся в памяти, сбрасываются на диск отсортированными запусками и затем сливаются в более крупные. Политик слияния две основные. При leveling на каждом уровне лежит один запуск, и каждое слияние переписывает его целиком. При tiering на уровне копится до T запусков, и они сливаются разом. Стоимость принято описывать тремя числами: сколько байт на диск уходит на байт данных пользователя (write amplification, WA), сколько блоков читает один запрос (read amplification) и во сколько раз файлы больше живых данных (space amplification).

Мой вопрос такой: как эти три величины, измеренные на настоящей реализации, соотносятся с оценками из учебников и документации, при каких условиях оценки точны и при каких нет, и как число бит на ключ в фильтре Блума определяет стоимость чтения. Вопрос закрыт, если выполнены три условия. Реализация проходит дифференциальные тесты, перебор точек падения и сверку с независимой моделью. Для WA, чтения и объёма названо условие, при котором оценка верна и при котором ошибается. Измеренная доля ложных срабатываний фильтра совпадает с формулой, а стоимость запроса с нулевым результатом выражается через число запусков и эту долю.

Постановка конкретная: запись 113 байт (ключ 8 байт, значение 100 байт), буфер 256 КиБ, блок 4 КиБ, T от 2 до 20, число записей N от 10^5 до 10^7. Кроме leveling и tiering я измеряю lazy leveling [7]: tiering на всех уровнях, кроме последнего.

2. Что уже известно

Структура предложена в статье O'Neil и соавторов [1]. Саму статью открыть из облачного контейнера не удалось (сетевая политика закрывает домены издателя и автора), поэтому о её содержании я сужу по документации RocksDB [4]. Документация RocksDB [4] называет классический leveling схемой, которая «минимизирует space amplification ценой read и write amplification», и пишет, что WA на уровень равна fanout в худшем случае и обычно меньше [8]. В Bigtable [2] те же идеи названы minor, merging и major compaction. Там же сказано, что Bloom-фильтры по SSTable избавляют большинство запросов к несуществующим ключам от обращений к диску. LevelDB [3] использует уровни по 10^L МБ и сливает по одному 2-мегабайтному файлу с перекрывающимися файлами следующего уровня.

Оценки WA по уровням я знаю из [4] и [8]. По аннотации [8] обычный анализ по худшему случаю завышает вставочную стоимость LevelDB в 1,8–3,5 раза, статью целиком я не читал. Фильтр Блума описан в [5]. Что вероятность ложного срабатывания при k хешах, m битах и n ключах равна $(1 - e^{-kn/m})^k$, я знаю как общепринятую формулу и проверяю её сам. По документации RocksDB [4] 9,9 бита на ключ дают около 1% ложных срабатываний. Стоимость нулевого запроса пропорциональна сумме вероятностей ложного срабатывания по запускам, а память фильтров выгодно распределять неравномерно по уровням: это из аннотации Monkey [6], саму статью я не открывал. Lazy leveling описан в аннотации Dostoevsky [7] как tiering на всех уровнях, кроме наибольшего, с WA ниже, чем у leveling, и теми же оценками чтения и объёма.

3. Что я сделал

3.1. Реализация

Библиотека написана на C++20 и занимает около 1040 строк. Запись идёт в журнал (WAL, у каждой записи CRC-32) и в буфер в памяти, при заполнении буфера он сбрасывается в SSTable. Файл таблицы состоит из блоков по 4 КиБ с CRC-32 на каждом, индекса, фильтра Блума и подвала. Фильтр использует двойное хеширование из одного 64-битного хеша, число хешей $k = \mathrm{round}(b \ln 2)$. Структура уровней записывается в манифест, который подменяется переименованием. Слияния выполняются синхронно в потоке записи, фоновых потоков нет, поэтому счётчики детерминированы.

Политики три: leveling, tiering и lazy leveling. Запуск, спущенный на пустой уровень, переименовывается без перезаписи (trivial move), как в RocksDB [4]. Эту оптимизацию можно выключить и измерить её вклад. Для фильтров можно задать число бит отдельно для каждого уровня. Надгробия при удалении исчезают, когда слияние охватывает всё, что глубже. Блочного кэша нет, индекс и фильтры всегда в памяти. Счётчики учитывают записанные байты сбросов, слияний и журнала, прочитанные блоки на запрос, опросы фильтра и ложные срабатывания.

3.2. Оценки, которые я вывел

Для WA при уникальных ключах я получил точную формулу (вывод мой, проверка ниже). Пусть $m = \lceil B/E \rceil$ записей в одном сбросе, $F = \lfloor N/m \rfloor$ сбросов. На уровень 1 приходит $A_1 = F$ пакетов, и после каждых T прихода уровень целиком уходит вниз, так что $A_{i+1} = \lfloor A_i/T \rfloor$. Приход номер j в leveling сливается с накопленным и пишет j пакетов, кроме первого, который при trivial move не пишется. Поэтому полный цикл из T приходов пишет $f(T)$ пакетов, где $f(a) = a(a+1)/2 - 1$, и записей слияний на запись пользователя получается

$$W_{\mathrm{lev}} = (1/N)\,\Sigma_{i \geq 1}\,(\lfloor A_i/T \rfloor\, f(T) + f(A_i \bmod T))\, m\, T^{i-1}. \tag{1}$$

К этому добавляются F m записей сбросов. Для tiering каждая запись пишется один раз на каждом уровне, который успел слиться:

$$W_{\mathrm{tier}} = (1/N)\,\Sigma_{i \geq 1}\,\lfloor A_i/T \rfloor\, m\, T^{i}. \tag{2}$$

В пределе полных циклов (1) даёт $(T+1)/2 - 1/T$ записей на запись на каждом уровне ниже первого. Это на $1/T$ меньше привычных $(T+1)/2$ за уровень и немного больше $T/2$. Эти оценки сравниваются с измерениями ниже. Для запросов с нулевым результатом оценка такая: если у фильтров k хешей и b бит на ключ, то стоимость в блоках равна $R{\cdot}(1 - e^{-k/b})^k$, где R означает число запусков, проверяющих ключ.

Для распределения бит по уровням я решаю задачу минимизации суммы вероятностей ложного срабатывания по запускам при фиксированной памяти. Метод множителей Лагранжа даёт, что вероятность у запуска уровня i должна быть пропорциональна n_i/r_i, где n_i записей на уровне и r_i запусков в нём. Уровни, для которых получается вероятность выше единицы, остаются без фильтра. Результат для большого уровня совпадает по духу с описанием Monkey [6], где вероятности тоже растут с уровнем, но проверить точное совпадение с [6] я не могу.

3.3. Корректность

Тесты (155 922 проверки) состоят из шести частей. Для CRC-32 проверяется стандартный вектор 0xCBF43926. Для фильтра проверяется отсутствие ложноотрицательных срабатываний и совпадение доли ложных с формулой на восьми значениях b. SSTable проверяется на чтение и на то, что испорченный байт в блоке даёт ошибку контрольной суммы. Дифференциальный тест сравнивает дерево с `std::map` на случайных потоках из 6000 операций (запись, удаление, чтение, диапазон до 30 записей, принудительный сброс, закрытие и открытие) для трёх политик, $T \in \{2, 3, 4, 10\}$ и четырёх началь-

ных значений, и после каждых 500 операций проверяет форму уровней. Тест оборванного журнала обрезает WAL в 30 случайных местах и требует, чтобы состояние было префиксом операций. Тест падений бросает исключение на контрольной точке (после записи в журнал, после каждого записанного файла таблицы, перед подменой манифеста и после неё, перед каждым удалением файла) и после повторного открытия сравнивает состояние с оракулом по подтверждённым операциям, а для той, где упали, допускает оба исхода. Выборочно проверяется до 400 точек на конфигурацию, полный перебор (все точки, 29 274 проверки) тоже проходит.

Чтобы убедиться, что тесты вообще что-то ловят, я включал пять намеренных поломок (таблица 1). Первая версия тестов падений пропускала две из них (удаление файлов до записи манифеста и усечение журнала до фиксации сброса): у меня не было контрольных точек между отдельными файловыми операциями, а набор из 80 ключей при буфере 4 КиБ почти не давал сбросов. Я добавил точки перед каждой операцией на диске и уменьшил буфер в тесте, и теперь все пять поломок ловятся.

Таблица 1. Намеренные поломки и то, какие части тестов их ловят.

поломка	какие части падают	проверок не прошло
при слиянии побеждает старая версия ключа	дифференциальный, падения	98 298
надгробия сбрасываются на любом уровне	дифференциальный, падения	45 884
удалить слитые файлы до записи манифеста	падения	28
усечь журнал до фиксации сброса	падения	121
фильтр не добавляет каждый 1009-й ключ	фильтр, дифференциальный, падения	2174

Структуру уровней и число записей при сбросах и слияниях я сверил с независимой моделью на Python, которая не делится с C++ кодом и работает на множествах ключей. Совпали записи сбросов, записи слияний и форма уровней ровно в 11 конфигурациях из 11 (три политики, $T = 4$ и 10, $N = 10^6$ уникальных ключей и $2{\cdot}10^6$ равномерных обновлений). Формулы (1) и (2) дали ровно те же числа записей, что и C++, на всех 56 прогонах leveling и tiering с уникальными ключами, включая $N = 10^7$.

В измерительной части была одна ошибка: в первом запуске параллельные процессы писали в один каталог и портили друг другу файлы, теперь каталог зависит от номера процесса. Кроме того, первая версия тестов прошла с первого раза, и это насторожило меня: отсюда и поломки из таблицы 1.

3.4. Окружение и как повторить

Все числа получены в облачном контейнере: Linux 6.18, Intel Xeon 2,1 ГГц, 4 ядра, 16 ГБ памяти, g++ 13.3.0, CMake 3.28.3, Python 3.13. Файлы лежат на tmpfs (`/dev/shm`), так что временные замеры показывают CPU и страничный кэш, диск в них не участвует. Счётчики (байты, блоки, опросы фильтра) от машины не зависят. Сверка с RocksDB сделана на версии 8.9.1 из пакета Ubuntu. Результаты в `results/` получаются командами:

```
cmake -S . -B build && cmake --build build -j4
./scripts/run_checks.sh   # тесты и поломки
./scripts/run_all.sh      # все эксперименты (около 20 минут на 4 ядрах)
python3 -I scripts/summary.py > results/summary.txt
python3 -I scripts/figures.py
```

4. Результаты

4.1. Фильтр Блума

Измеренная доля ложных срабатываний совпадает с формулой $(1 - e^{-kn/m})^k$ на всём диапазоне b от 1 до 20 (рис. 1): отношение измеренного к формуле лежит между 0,996 и 1,001 на $2{\cdot}10^7$ проб, ложноотрицательных нет. Приближение $e^{-b\ln^2 2}$ хуже при малых b: при $b = 1$ оно даёт 0,619 против измеренных 0,631, потому что k приходится округлять до целого. При $b = 10$ фильтр с $k = 7$ даёт 0,82%, что согласуется с «около 1%» для 9,9 бита из документации RocksDB [4].

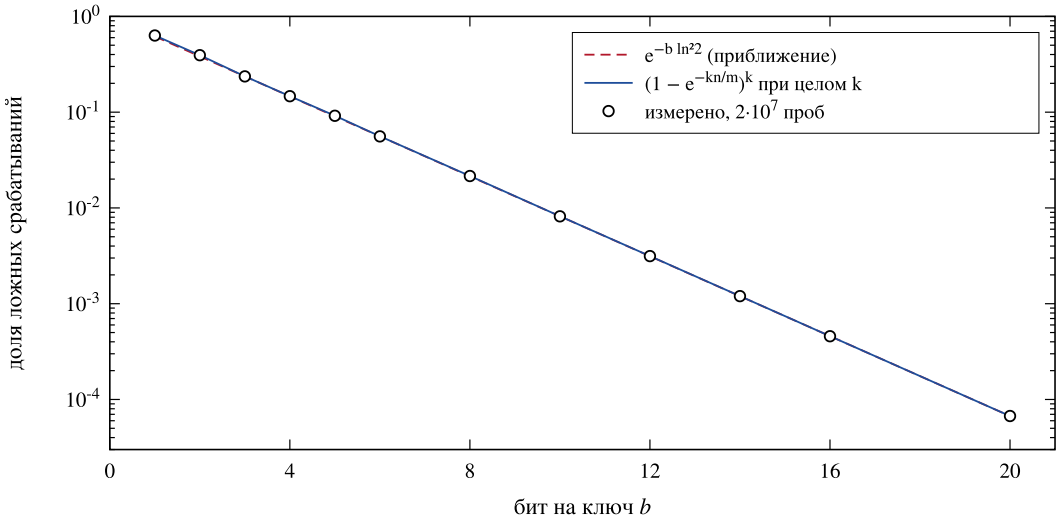
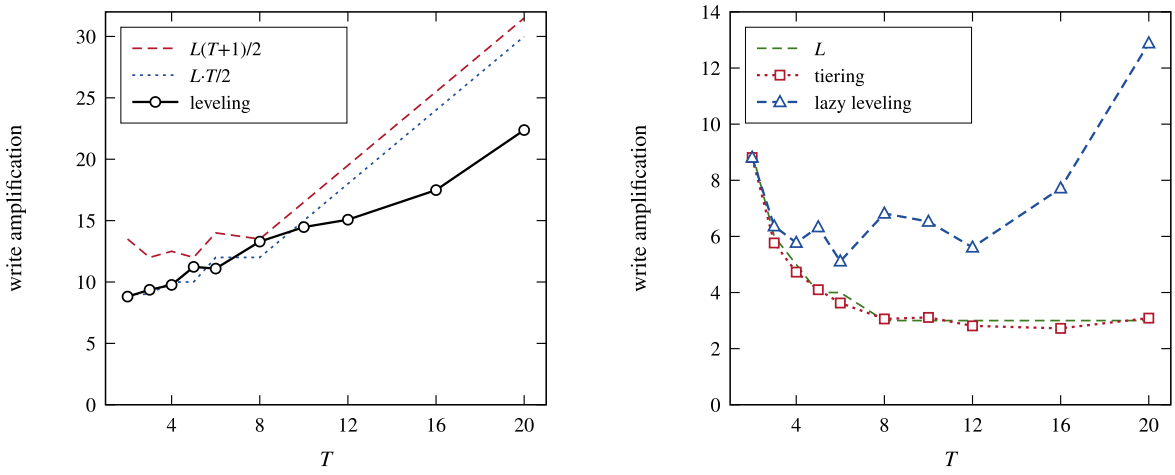


Рис. 1. Доля ложных срабатываний фильтра Блума при $n = 10^6$ ключей. Маркеры показывают измерение по $2{\cdot}10^7$ пробам заведомо отсутствующих ключей, сплошная линия показывает формулу для целого $k = \mathrm{round}(b \ln 2)$, пунктир даёт приближение $e^{-b\ln^2 2}$.

4.2. Write amplification

Рис. 2 и таблица 2 дают WA при $N = 10^6$ уникальных ключей. Для leveling измеренная WA лежит между $L{\cdot}T/2$ и вдвое ниже $L{\cdot}T$: измеренное составляет 0,48–0,49 от $L{\cdot}T$ при $T = 2$, 4 и 10 и 0,37 при $T = 20$, что согласуется с завышением в 1,8–3,5 раза у [8]. Оценка $L{\cdot}T/2$ попадает в измерение в пределах 12% при $T \le 10$ и завышает при $T = 20$ (30 против 22,4), потому что последний уровень при $N = 10^6$ заполнен на несколько процентов. Оценка $(T{+}1)/2$ за уровень завышает сильнее, на 14–53%. WA я считаю как байты файлов на байт пользователя, и около 6% к ней добавляют заголовок записи, индекс и фильтр. Для tiering WA близка к числу уровней L при $T \ge 8$ (3,06–3,11 при $L = 3$) и ниже L при $T = 4$ (4,73 против 5), потому что последний уровень тоже не заполнен. При $T = 2$ все три политики совпадают (8,81), как и должно быть.



(а) leveling

(б) tiering и lazy leveling

Рис. 2. Write amplification при $N = 10^6$, уникальные ключи. (а) Leveling: измерение и оценки $L(T+1)/2$ и $L \cdot T/2$ при L по ёмкости уровней. (б) Tiering и lazy leveling; пунктир показывает число уровней L.

Таблица 2. WA при $N = 10^6$, уникальные ключи. L означает число уровней по ёмкости. «Худший» обозначает $L \cdot T$, оценку «fanout за уровень».

политика	T	L	измерено	$L \cdot T/2$	$L(T+1)/2$	худший
leveling	2	9	8,81	9,0	13,5	18
leveling	4	5	9,76	10,0	12,5	20
leveling	10	3	14,47	15,0	16,5	30
leveling	20	3	22,37	30,0	31,5	60
tiering	4	5	4,73			$L = 5$
tiering	10	3	3,11			$L = 3$
tiering	20	3	3,09			$L = 3$
lazy leveling	4	5	5,78		между tiering и leveling	
lazy leveling	10	3	6,53			
lazy leveling	20	3	12,88			

Из таблицы видно и главное ограничение учебных оценок: при фиксированном N WA не гладкая функция T. Для $T = 5$ и 6 при $N = 10^6$ получается 11,23 и 11,09, то есть рост T WA не увеличил. Причина видна из формулы (1): стоимость определяется цифрами числа сбросов F в системе счисления с основанием T. Пока число пакетов на последнем уровне $A_i \bmod T$ мало, этот уровень почти не переписывается. Рис. 3 показывает пилу WA(N) по формуле и измеренные точки, которые ложатся на неё. Отношение WA к $L(T+1)/2$ при N от 10^5 до 10^7 меняется от 0,68 до 1,09 при $T = 10$ и от 0,72 до 1,02 при $T = 4$. Верхний край достигается перед появлением нового уровня, нижний сразу после него, когда новый уровень почти пуст, а L уже выросло. При $N = 10^7$ формула даёт 20,21 при $T = 10$ и 15,13 при $T = 4$, измерено 20,22 и 15,13 (разница в сотых от того, что я пересчитываю записи в байты файлов постоянным множителем).

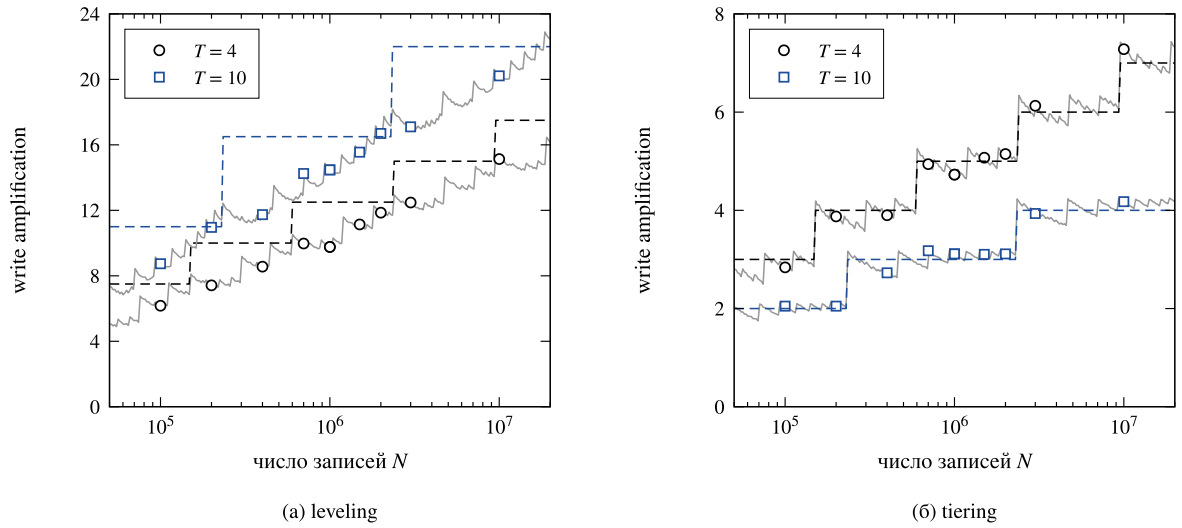


Рис. 3. WA в зависимости от числа записей. Серая пила получена из формул (1) и (2), маркеры показывают измерение, чёрный пунктир даёт оценку $L(T{+}1)/2$ для leveling и L для tiering. (а) Leveling. (б) Tiering.

Trivial move вносит заметный вклад: при $N = 10^6$ выключение этой оптимизации поднимает WA leveling с 9,76 до 11,25 при $T = 4$ (на 15%) и с 14,47 до 14,84 при $T = 10$ (на 3%). Оценки, которые считают, что каждое поступление на уровень переписывается, теряют именно эту разницу.

4.3. Обновления и перекос

Формулы (1) и (2) предполагают уникальные ключи. Когда ключи повторяются, слияния выбрасывают старые версии, и WA падает (рис. 4, таблица 3). Я писал $2{\cdot}10^6$ записей равномерно по U ключам и по Zipf на 10^6 ключах с показателями 0,8, 0,99 и 1,2. Для leveling при $T = 10$ WA равна 9,5 при $U = 20\,000$ и 15,1 при $U = 10^6$, а с Zipf 1,2 всего 2,2 (различных ключей 6% от числа записей). Для tiering при $T = 10$ WA падает до 0,49 при Zipf 1,2: большинство перезаписей гасятся ещё в буфере, и в файлы уходит меньше данных, чем пришло.

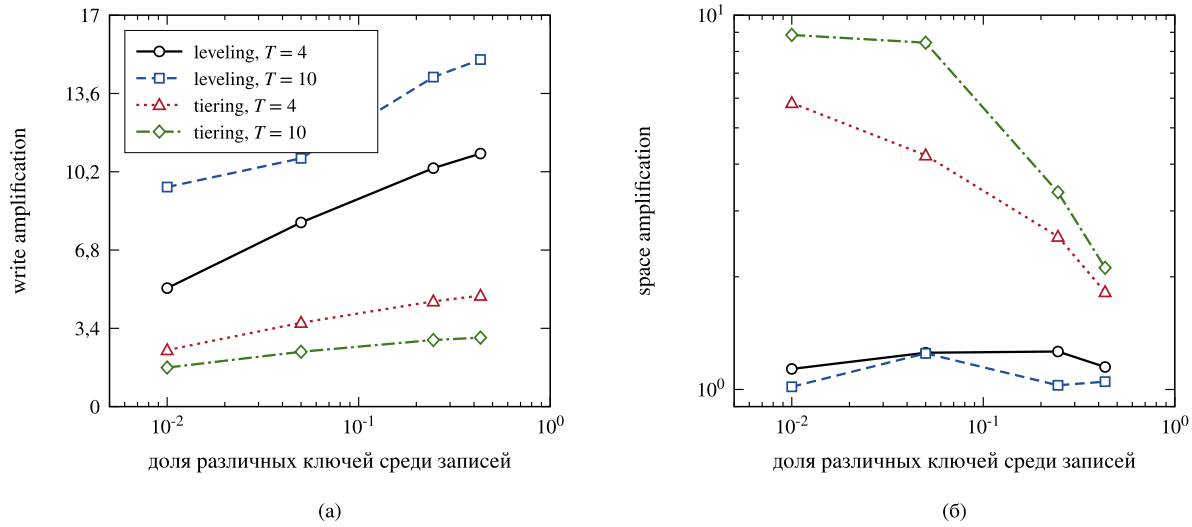


Рис. 4. Равномерные обновления, $2{\cdot}10^6$ записей, $U \in \{2{\cdot}10^4, 10^5, 5{\cdot}10^5, 10^6\}$. По горизонтали доля различных ключей среди записей. (а) WA. (б) Space amplification, логарифмическая шкала.

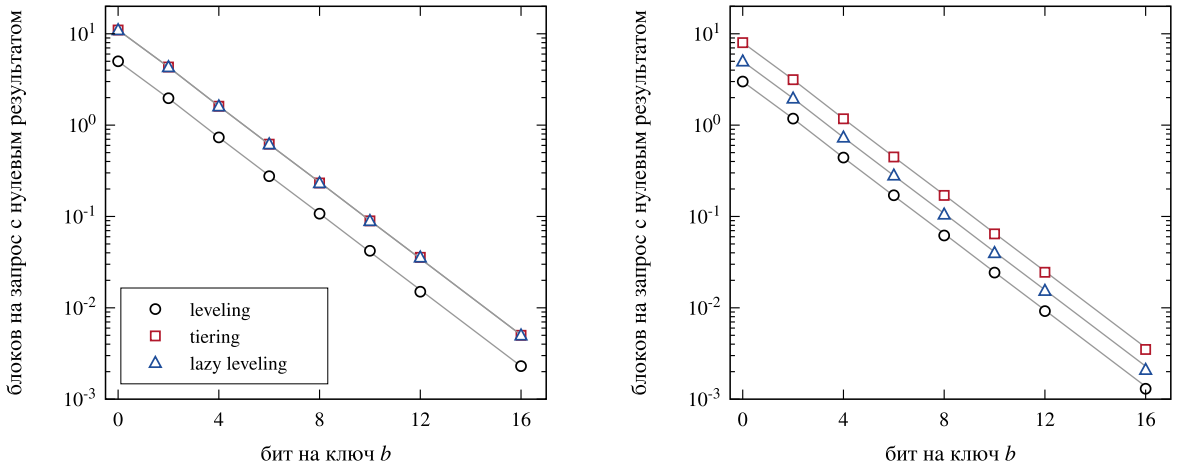
Таблица 3. WA и space amplification при $2{\cdot}10^6$ записях. «Различных» означает долю различных ключей среди записей.

нагрузка	различных	leveling $T = 10$		tiering $T = 10$		tiering $T = 4$	
		WA	объём	WA	объём	WA	объём
равномерная, $U = 2{\cdot}10^4$	0,010	9,54	1,02	1,70	8,86	2,45	5,83
равномерная, $U = 10^5$	0,050	10,78	1,25	2,38	8,44	3,64	4,22
равномерная, $U = 10^6$	0,432	15,07	1,05	3,00	2,11	4,82	1,82
Zipf 0,8	0,292	11,94	1,04	2,32	2,05	3,75	1,57
Zipf 0,99	0,178	7,04	1,32	1,44	1,75	2,23	1,36
Zipf 1,2	0,064	2,18	1,08	0,49	1,37	0,68	1,49

Space amplification показывает, где ломается учебная оценка. Для leveling она не превышает 1,32 ни при одной нагрузке. Оценка $1 + 1/T$ попадает при $T = 4$ (1,254 при $U = 10^5$ против 1,25) и занижает при $T = 10$ (1,249 против 1,1). Для tiering оценка «порядка T» опирается на то, что последний уровень содержит $T - 1$ копий данных, а выше лежит геометрически убывающий хвост. Если ключей мало и они все помещаются в один запуск, хвост перестаёт убывать, потому что запуски всех уровней содержат почти весь набор. Измерено 8,86 при $T = 10$ и 5,83 при $T = 4$, и второе число выше T: в дереве девять запусков, каждый до 20 000 записей из тех же 20 000 ключей. С ростом доли различных ключей объём падает до 2,1 и 1,8.

4.4. Чтение

Рис. 5 показывает среднее число блоков, прочитанных запросом с нулевым результатом, против b. Измерение ложится на модель «число запусков, умноженное на вероятность ложного срабатывания» с наибольшим относительным отклонением 8,4% на 42 клетках (при $b = 16$ стоимость порядка 10^{-3} блока, и 50 000 запросов дают лишь десятки событий). При $b = 0$ запрос читает по блоку из каждого запуска, и число запусков можно прочесть по этому значению: 5 у leveling при $T = 4$, 3 при $T = 10$, 11 и 8 у tiering.



(а) $T = 4$ (б) $T = 10$

Рис. 5. Запросы с нулевым результатом, $N = 10^6$, 50 000 запросов на точку. Маркеры показывают измерение, серые линии дают модель «число запусков × вероятность ложного срабатывания». У tiering и lazy leveling при $T = 4$ оба дерева в момент замера имеют одинаковую форму, и точки совпадают.

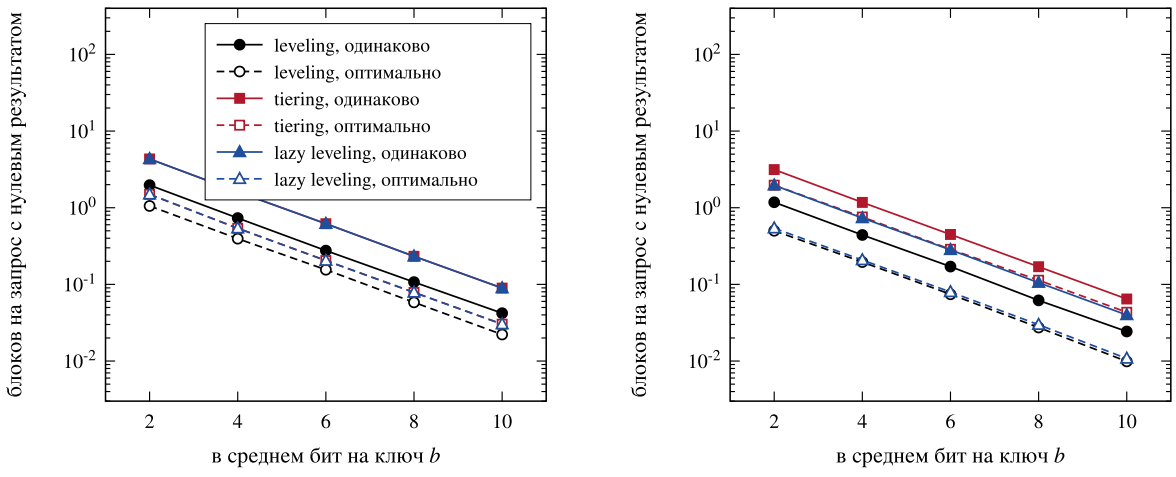
Таблица 4. Чтение при $b = 10$, $N = 10^6$, блоков на запрос. «Диапазон» означает короткий скан на 100 записей.

политика	T	запусков	нулевой	модель	существующий	диапазон
leveling	4	5	0,0421	0,0410	1,028	7,8
leveling	10	3	0,0243	0,0246	1,016	5,7
tiering	4	11	0,0895	0,0901	1,075	14,0
tiering	10	8	0,0645	0,0655	1,042	10,9
lazy leveling	10	5	0,0399	0,0410	1,030	7,8

Число запусков, а с ним и стоимость чтения, тоже зависит от N, и оценка «$(T - 1)$ запусков на уровень» для tiering описывает худший момент цикла. При $b = 10$ и $N = 10^6$ у tiering с $T = 10$ запусков 8 и стоимость 0,065, а с $T = 12$ запусков 24 и стоимость 0,197, хотя «худший» случай для $T = 10$ был бы 27. Запрос существующего ключа при $b = 10$ читает 1,02–1,08 блока: один блок в запуске, где ключ есть, плюс ложные срабатывания выше. Короткий скан на 100 записей читает примерно число запусков плюс 2,8 блока (сами записи занимают $100 \cdot 113/4096 \approx 2{,}8$ блока): 5,7 при трёх запусках, 10,9 при восьми. После обновлений (200 000 различных ключей из $2 \cdot 10^6$ записей) запросов к существующим ключам дерево читает 1,005–1,081 блока, а нулевых 0,017–0,165.

4.5. Одинаковые фильтры или оптимальное распределение

Рис. 6 сравнивает одинаковое число бит на ключ на всех уровнях с распределением, которое минимизирует сумму вероятностей ложного срабатывания при той же общей памяти (она совпадает с точностью 2%). Выигрыш в блоках на запрос с нулевым результатом составляет 1,8–1,9 раза для leveling при $T = 4$, 2,3–2,5 при $T = 10$, 2,9–3,0 для tiering при $T = 4$, 1,5–1,6 при $T = 10$ и 3,5–3,7 для lazy leveling при $T = 10$. Выигрыш почти не зависит от b в диапазоне 2–10. Запрос существующего ключа выигрывает ещё сильнее при малых b (при $b = 2$ у tiering с $T = 4$ 1,42 блока вместо 4,58). Выигрыш измерен на одном N, и положение уровней в момент замера сильно на него влияет.



(а) $T = 4$ (б) $T = 10$

Рис. 6. Запросы с нулевым результатом при одинаковой памяти фильтров. Сплошные линии показывают одинаковое число бит на всех уровнях, штриховые показывают оптимальное распределение (вероятность ложного срабатывания пропорциональна размеру запуска).

4.6. Запись против чтения

Рис. 7 переносит всё на одну плоскость: по горизонтали WA, по вертикали стоимость запроса с нулевым результатом при $b = 10$, по точке на каждое T от 2 до 20 для каждой политики. Leveling даёт самое дешёвое чтение (0,024–0,058 блока) за 9–22 записи на запись. Tiering доходит до WA 2,7–3,1, и чтение при этом стоит 0,06–0,22 блока. Lazy leveling лежит между ними, и при $N = 10^6$ точка lazy leveling с $T = 10$ (WA 6,53, чтение 0,040) лучше точки leveling с $T = 4$ (9,76 и 0,042) по обоим осям. Вывод, что lazy leveling в целом превосходит leveling, из одной точки сделать нельзя. Из-за пилообразности WA и числа запусков от T соседние точки одной политики прыгают, и кривая не монотонна.

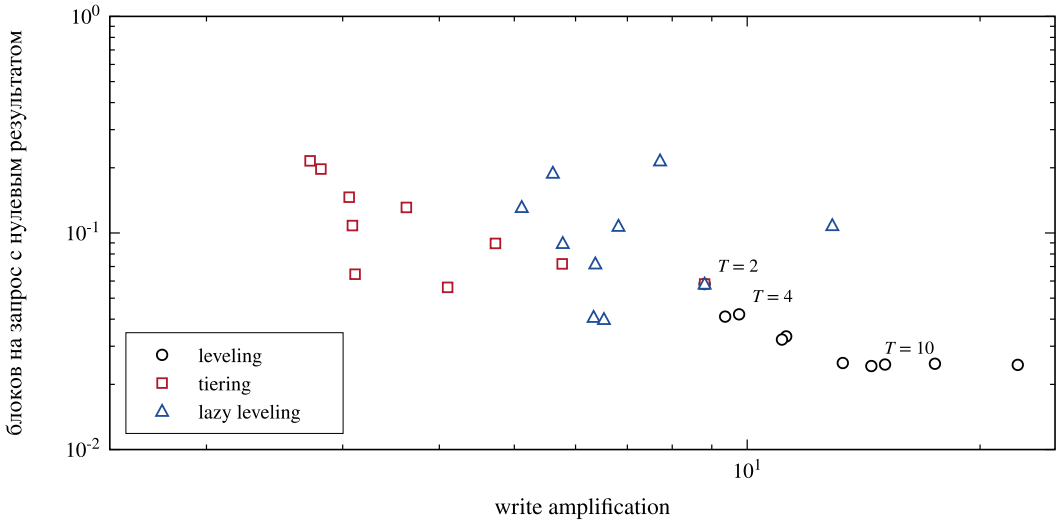


Рис. 7. WA и стоимость запроса с нулевым результатом при $b = 10$, $N = 10^6$, T от 2 до 20 для каждой политики. Подписаны точки leveling.

4.7. Сравнение с RocksDB

Для внешней проверки я собрал тест на RocksDB 8.9.1 с теми же размерами: буфер 256 КиБ, ключ 8 байт, значение 100, 10 бит на ключ, без сжатия, без WAL, ёмкость первого уровня $T{\cdot}256$ КиБ и множитель T (рис. 8, таблица 5). Совпадения конфигураций нет: RocksDB сливает отдельные файлы, мой leveling сливает уровень целиком, и RocksDB держит промежуточный уровень L0 с порогом в четыре файла. Универсальный стиль (tiering-подобный) даёт WA 3,4–5,4 и совпадает по порядку с моим tiering (3,1–4,2 при $T = 10$). Классический leveling RocksDB даёт 4,3–6,2 (4,7–5,0 при $N = 10^7$), а мой полный leveling 14,5 при 10^6 и 20,2 при 10^7. Статистика самого RocksDB (`rocksdb.stats`) показывает, что почти всё ниже уровня L2 попало туда простым перемещением файлов без перезаписи. Подробнее этот эффект я не исследовал. Поэтому сверка подтверждает порядок величин для tiering и показывает, что учебная оценка $L{\cdot}T/2$ для leveling не переносится на систему с частичными слияниями, но не проверяет мою модель leveling.

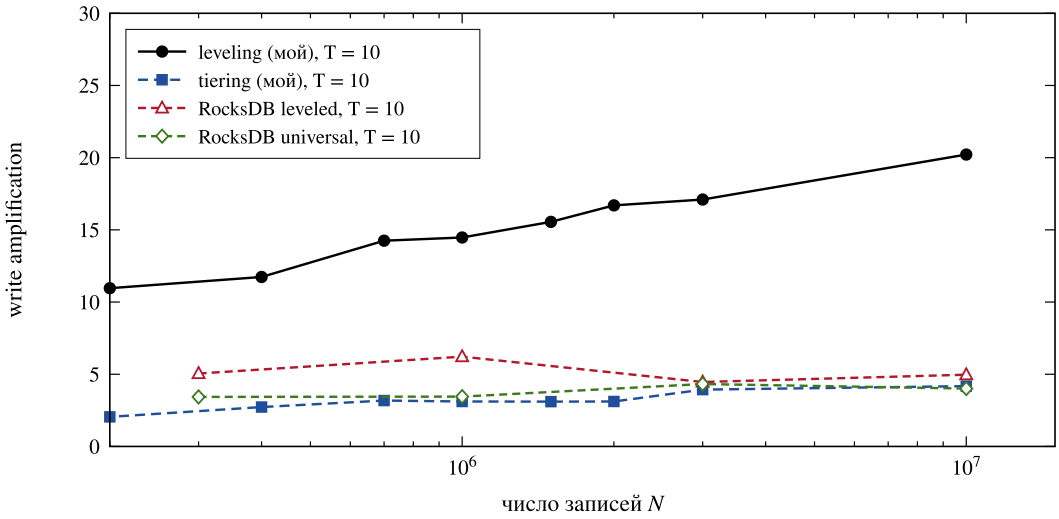


Рис. 8. WA в зависимости от N, $T = 10$, уникальные ключи. Мои leveling и tiering против RocksDB 8.9.1 (классический leveling со статическими размерами уровней и универсальный стиль).

Таблица 5. WA RocksDB 8.9.1, $T = 10$. Два запуска одной и той же конфигурации в моих прогонах расходились до 25% (фоновые потоки), приведён один из них.

N	leveled	leveled, dynamic	universal	мой leveling	мой tiering
10^6	6,22	6,28	3,45	14,47	3,11
$3{\cdot}10^6$	4,46	6,78	4,32	17,10	3,94
10^7	4,97	7,83	4,01	20,22	4,18

4.8. Время

Таблица 6 даёт время на tmpfs в трёх повторах подряд. Загрузка 10^6 записей занимает 10,9–11,3 с при leveling, 2,9–3,2 с при tiering и 4,9–5,6 с при lazy leveling, то есть порядок политик совпадает с порядком WA. Нулевой запрос стоит 0,46–0,56 мкс при leveling и 1,16–1,22 мкс при tiering: он тратит время только на опросы фильтров. Запрос существующего ключа стоит 11–14 мкс почти независимо от политики, потому что обращение к блоку (чтение 4 КиБ, проверка CRC, разбор) на порядок дороже фильтров. Эти числа получены на страничном кэше, диск в них не участвует.

Таблица 6. Время при $T = 10$, $N = 10^6$, $b = 10$, минимум и максимум по трём повторам.

политика	загрузка, с	нулевой запрос, нс	существующий, нс	скан на 100, мкс
leveling	10,9–11,3	457–562	11 387–14 278	74–76
tiering	2,9–3,2	1162–1222	11 355–14 316	137–153
lazy leveling	4,9–5,6	760–803	12 468–14 703	91–106

5. Где это может быть неверно

Слияния у меня синхронные, и в измерениях нет ни фоновых потоков, ни задержек записи при отставании слияний. Именно на этом реальные системы теряют пропускную способность, и мои измерения времени этого не показывают. Данные лежат на tmpfs, и fsync не вызывается, поэтому проверка падений моделирует остановку процесса в контрольных точках, а потерю незаписанных страниц при отказе питания она не проверяет. Блочного кэша нет: кэш изменил бы число блоков на запрос.

Значения фиксированной длины 100 байт, ключи случайные 64-битные. Для возрастающих ключей слияния почти не нужны, и WA была бы близка к 1, но я это не мерил. Формулы (1) и (2) верны только для уникальных ключей. Для обновлений и перекоса у меня нет формулы, только измерения при одном N и одном размере буфера. Все эксперименты идут при буфере 256 КиБ, и поведение при другом соотношении буфера к записи может быть иным.

Распределение бит по уровням посчитано по форме дерева в конце загрузки и затем использовано в повторной загрузке; на растущем дереве оно было бы чуть хуже. Сверка с RocksDB грубая: конфигурации не совпадают, слияния асинхронные, и эффект перемещения файлов я не разбирал. Статьи [1], [5], [6], [7] и [8] я не читал и опираюсь на их пересказы и аннотации, поэтому формулировки из них нужно сверить с оригиналами.

Фильтр проверен при $n = 10^6$ и случайных ключах. На малых запусках (сотни ключей), на ключах с общей структурой и на блочных фильтрах, где проба укладывается в одну линию кэша, формула может не работать.

6. Открытые вопросы

Дальше я бы реализовал слияние отдельными файлами с выбором наименее перекрытого файла, чтобы объяснить разрыв с RocksDB (14–20 против 4–6) на собственной модели, и измерил его при перемешанных и возрастающих ключах. Нужны фоновые слияния с задержками записи, удаления с надгробиями при большой доле удалённых ключей, блочные фильтры и блочный кэш. Полезно сравнить мои цифры lazy leveling с результатами Dostoevsky [7] на тех же N, но для этого нужно прочитать саму статью и взять её параметры.

Литература

[1] O'Neil P., Cheng E., Gawlick D., O'Neil E. The Log-Structured Merge-Tree (LSM-Tree) // Acta Informatica 33(4), 1996, p. 351–385. Оригинал открыть не удалось, содержание знаю по [4] и поисковой выдаче.

[2] Chang F., Dean J., Ghemawat S., Hsieh W. C., Wallach D. A., Burrows M., Chandra T., Fikes A., Gruber R. E. Bigtable: A Distributed Storage System for Structured Data // OSDI 2006. Читал, разделы 5.3 и 5.4 и подраздел «Bloom filters» (копия из репозитория papers-we-love).

[3] LevelDB, документ doc/impl.md (Google, github.com/google/leveldb). Читал целиком.

[4] RocksDB wiki (Meta/Facebook, github.com/facebook/rocksdb/wiki): страницы Compaction, Leveled-Compaction, Universal-Compaction, RocksDB-Bloom-Filter. Читал, версия вики от февраля 2026 года.

[5] Bloom B. H. Space/time trade-offs in hash coding with allowable errors // Communications of the ACM 13(7), 1970, p. 422–426. Библиографию проверил по поисковой выдаче, текст не читал.

[6] Dayan N., Athanassoulis M., Idreos S. Monkey: Optimal Navigable Key-Value Store // SIGMOD 2017. Знаю по аннотации и пересказу в поисковой выдаче.

[7] Dayan N., Idreos S. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging // SIGMOD 2018. Знаю по аннотации и пересказу в поисковой выдаче.

[8] Lim H., Andersen D. G., Kaminsky M. Towards Accurate and Fast Evaluation of Multi-Stage Log-structured Designs // FAST 2016, p. 149–166. Знаю по аннотации и по ссылке из [4].